AI Systems Performance Engineering

Chapter 6 — GPU Architecture, CUDA Programming, and Maximizing Occupancy

Part V structure preview (draft)

Tianqin Cai
Applied Scientist
AWS

Internal Training for Applied Scientists on AI Infrastructure

August 15, 2026

Part V — Debugging Correctness: NVIDIA Compute Sanitizer

No more blind-tuning block sizes — next: compute ceiling or memory ceiling? (roofline, after a correctness detour)

Memory

- **memcheck** — out-of-bounds / misaligned accesses (global, local, shared), HW exceptions, device leaks; `--check-device-heap`.
- **initcheck** — reads of uninitialized global memory (classic cause: a **missing H2D copy**).

Concurrency

- **racecheck** — shared-memory hazards: **WAW / WAR / RAW** ⇒ nondeterministic behavior.
- **synccheck** — invalid sync primitives, mismatched barriers ⇒ deadlocks, inconsistent state.

- CLI: `compute-sanitizer [--tool <name>] <app>`; filter with `--kernel-name`, annotate with **NVTX** (`--nvtx yes`).
- Book tip: run it in **CI** with `--error-exitcode`.[5]

Plain English: "it works on my run" means nothing at 100,000 threads — the sanitizer is your seatbelt.

Roofline Tells Us Which Resource Is the Limit

- **Arithmetic intensity** = FLOPs per byte moved to/from HBM.
- Two ceilings: **flat** compute roof (∼80 TFLOP/s FP32) and **sloped** memory roof (∼8 TB/s), crossing at the **ridge point** ≈ **10 FLOPs/byte** (80T ÷ 8T).
- Worked example: $C = A + B$ loads 8 B, adds once, stores 4 B ⇒ 1 FLOP / 12 B = **0.083 FLOPs/byte** — >100× left of the ridge: hopelessly **memory-bound**.
- Book note: LLMs are **both** — **prefill** tends compute-bound, **decode** tends memory-bound (Ch. 15–18).

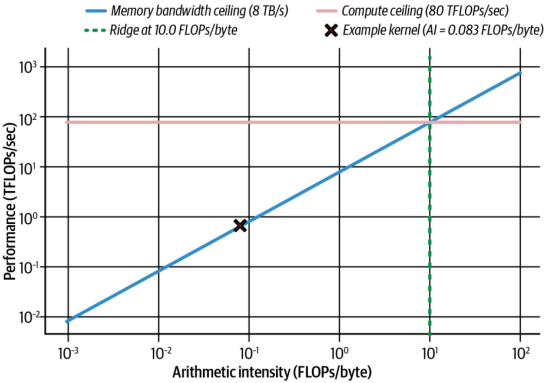

Figure 6-21. Roofline for a Blackwell-class GPU; our kernel sits at 0.083 FLOPs/byte

Plain English: the roofline asks first: is this kernel **starving for math, or for data?**[4]

Moving Right on the Roofline: Lower Precision Pays Twice

- To escape the memory-bound region, **do more work per byte**: reuse data on-chip, fuse ops — or simply **shrink the bytes**.
- One 128-byte transaction carries **32 FP32 = 64 FP16 = 128 FP8 = 256 FP4** values. FP32 $\rightarrow$ FP16 instantly **doubles** FLOPs/byte; FP8 doubles throughput and halves memory again vs. FP16.
- Blackwell adds native **FP8 and FP4 Tensor Cores**, plus **hardware decompression**: weights stored compressed in HBM (even beyond FP4 — 4-bit/2-bit schemes) are expanded on the fly and cast to FP16/FP32 for accurate accumulation.
- Net: **effectively more memory bandwidth** — an architectural edge for memory-bound **token generation**.

The takeaway

Precision is a **bandwidth** optimization as much as a compute one: halve the bytes $\Rightarrow$ double the arithmetic intensity $\Rightarrow$ move toward the compute roof.

Profiling Workflow: Diagnose, Fix, Re-measure

- Memory-bound signature in **ncu**: **high DRAM utilization + low ALU utilization** — warps stalled on memory. Also watch **global load efficiency** dropping.
- **nsys** timeline shows idle stretches between kernels = GPU waiting on transfers.
- Expected overlap but see sequential copy-then-kernel? Two usual suspects: an unwanted **default-stream sync**, or a **missing pinned-memory buffer** — without pinned memory, `cudaMemcpyAsync` **cannot overlap** with kernels.
- Newer ncu niceties: **range replay**, source correlation, launch-stack size metric.
- After fixes: idle gaps vanish, **memory pipe utilization** climbs toward peak.

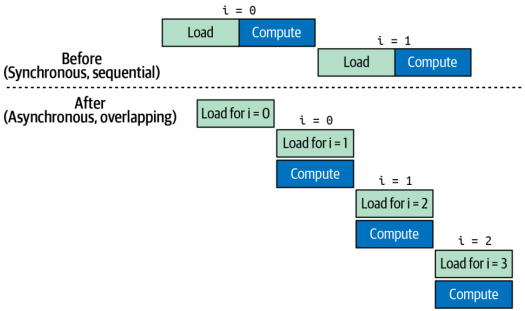

Figure 6-22. Sync (sequential) vs. async (overlapped) transfers + compute *Always measure after each change — profilers confirm whether stalls actually dropped.*

Key Takeaways (the Book's Eight)

- **SIMT**: 32-thread warps in lockstep; many warps in flight = latency hidden.
- **Hierarchy**: threads → blocks ($\leq$1,024) → grids; minimize `__syncthreads()` barriers.
- **Occupancy vs. limits**: multiples of 32; per-SM: 64 warps, 32 blocks, 228 KB shared, 255 regs/thread.
- **Launch params**: 256 threads/block to start; grid = ceil(N/256); tune via profiling.
- **Async memory**: `cudaMallocAsync` + streams + pools; PyTorch's caching allocator ditto.
- **Memory ladder**: reuse in registers/shared/L1; coalesce into L2/HBM.
- **Unified Memory**: prefetch + advise to kill surprise stalls.
- **Roofline**: FLOPs/byte picks your battle; FP16/FP8/FP4 + hardware decompression move you right; **TMEM + UMMA** can shift kernels memory- → compute-bound.

Conclusion and What's Next

This chapter in one sentence

Make the GPU **busy** (occupancy), make data **close** (memory hierarchy), and let the **roofline** tell you which battle to fight next.

- The book's closing caveat: **max occupancy is not always optimal** — with enough **ILP** per thread, moderate/low occupancy can win; sometimes **fewer threads with more registers each** is faster. **Always benchmark.**
- Coming next in the book: warp divergence & memory-access tuning (Ch. 7–8), arithmetic intensity (Ch. 9), TMA & warp specialization (Ch. 10).
- My second talk — **Chapter 12** — builds on this: once single kernels are fast, how do we **orchestrate** them so the GPU never waits for the CPU?

Questions?

References & Further Reading

1. C. Fregly. *AI Systems Performance Engineering* (O'Reilly, 2026), Chapter 6.
2. NVIDIA. *Blackwell Tuning Guide* and *Blackwell Architecture Whitepaper* — SM limits, dual-issue schedulers, TMEM/tcgen05.
3. NVIDIA. *CUDA C++ Programming Guide* — thread hierarchy, Unified Memory, occupancy API, `__launch_bounds__`, stream-ordered allocation, PTX/fatbin.
4. S. Williams, A. Waterman, D. Patterson. *Roofline: An Insightful Visual Performance Model for Multicore Architectures*. CACM 52(4), 2009.
5. NVIDIA. *Compute Sanitizer User Guide* — memcheck, racecheck, initcheck, synccheck.
6. NVIDIA. *Nsight Systems* and *Nsight Compute* documentation — `nsys profile`, `ncu --section SpeedOfLight`, NVTX.
7. Book code and benchmarks: the book's GitHub repository (per-architecture results for the illustrative tables in this deck).